Time Is the Baseline to Beat: Lead Time over a Timer as the Test for VLA Failure Monitors

Author Names Omitted for Anonymous Review. Paper-ID [add your ID here]

***Abstract*—Runtime failure monitors for vision-language-action (VLA) policies are often evaluated under success-terminated, time-limited protocols, where a failed episode is one that reaches its time limit. There, elapsed time is a strong failure signal: on RoboCasa365, a timer calibrated at a 5% false-positive rate (FPR) flags every failure, and a model that sees only elapsed time separates failures as well as one that reads the policy. Without a duration-only comparator, even timeliness metrics cannot show what a monitor adds. We measure how much earlier than a per-task timer a monitor alarms at the same target FPR, and fuse the two by an OR rule whose split-conformal calibration bounds the FPR for a path fixed independently of the calibration successes. On 760 test rollouts of one VLA on 38 tasks, three-seed ensembles of the SAFE failure score and of the sequence detector Hide-and-Seek add 0.030 and 0.071 of the time limit to the timer; at matched false alarms they shrink to 0.015 and 0.056, neither clearly above zero. Input ablations that see only elapsed time or another rollout's features add at most 0.012. On unseen tasks, where no per-task timer exists, Hide-and-Seek still adds 0.127 over a global timer (0.051 at matched false alarms), SAFE about as much as random noise. The ordering holds on recorded rollouts of a second VLA, and on FIPER's benchmark the test credits FIPER. Stage 2 tests the second VLA on new rollouts and whether earlier alarms raise success through retry.**

I. Introduction

Vision-language-action (VLA) policies [13, 5] fail often enough that a deployed robot must notice when an attempt is going wrong. Runtime monitors map the policy's hidden state to a failure score [8], or score how unusual its embeddings or actions are relative to successful rollouts [30, 24]. They are compared with each other on ROC-AUC, balanced accuracy and, increasingly, timeliness [24, 22], but rarely with what the robot knows for free: how long the attempt has been running.

Under success-terminated, time-limited protocols such as those of RoboCasa365 [20] and LIBERO [15], an episode ends at the first success or at a fixed time limit, so a failure is, by construction, an episode that ran out of time. In RoboCasa365 every one of our failed rollouts ran to its limit, and on our validation set a timer at 90% of the limit flags all of them at 4.7% FPR. Elapsed time also explains separation: an LSTM trained on zero inputs, which sees only elapsed time, matches the ROC-AUC of the same LSTM trained on the policy's features (0.888 against 0.878). On such benchmarks, a monitor can add only one thing over a timer: an earlier alarm at the same false-alarm rate (Fig. 1), whose value Stage 2 tests by acting on it.

We therefore evaluate monitors by their *warning lead* over a per-task timer that alarms once an attempt has run longer than the task's successful attempts usually take. We hypothesize that the policy's internal state carries warning that elapsed time does not: fused with this timer, a learned failure score should raise the mean lead at the same FPR, and the gain should vanish when the score cannot see the rollout. Our contributions are:

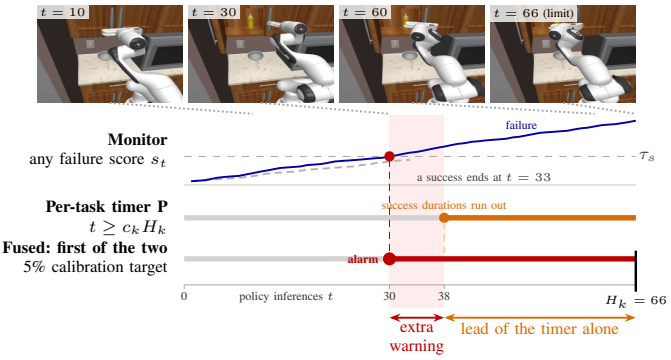


Fig. 1. In success-terminated, time-limited benchmarks every failed rollout runs to its time limit H_k, so a per-task timer that alarms once an attempt outlasts the task's successes flags all failures. A monitor helps only if it alarms *earlier*. We fuse any monitor with the timer (first alarm wins), calibrate both at a 5% false-alarm target, and measure the extra warning over the timer. Example: a failed XR-1 *OpenCabinet* rollout, monitor score from SAFE.

- **A duration-only comparator for timeliness metrics:** warning lead over a timer at matched FPR, on known and unseen tasks, with input ablations that separate rollout-specific evidence from learned elapsed time (Sec. III, IV).
- **Time-aware fusion:** an OR rule that fuses any monitor with a timer, whose split-conformal calibration bounds the FPR in finite samples for a path fixed independently of the calibration successes (Proposition 1).
- **An evaluation on 38 RoboCasa365 tasks.** As three-seed ensembles, SAFE adds 0.030 of the time limit over the per-task timer and Hide-and-Seek [22] 0.071, a warning that develops during the rollout (Sec. VI). On XR-1's unseen tasks, only Hide-and-Seek's gain carries over; the ordering holds on a second VLA, and on FIPER's benchmark the test credits FIPER.

Most failures we annotated had reached no irreversible state by the limit, which is why lead, not recall, is the quantity to report.

II. Related Work

Failure detection from policy internals. SAFE learns a failure score from a VLA's last-layer features and thresholds it with a functional conformal band [8]; we reuse its implementation. Follow-ups change the training data [23], labels [22, 18], or policy [26]. FIPER notes that accumulated scores flag failures mostly because failed rollouts are longer [24], and ActProbe feeds the normalized timestep to its model [11], but none reports an elapsed-time detector. SAFE identifies the duration shortcut and removes it by evaluating rollouts at equal length (its App. B.5) and, on its real robot, by running every rollout to a fixed horizon [8]. These controls suit fixed-horizon evaluation; under success-terminated execution, duration is a shortcut [7] for offline metrics but legitimate online evidence, much as time limits make the remaining time part of the agent's state in reinforcement learning [21].

Action-space and success-only detectors. STAC [1] and VLA-FAIL [25] compare overlapping action chunks, and ActProbe [11] feeds chunk consistency, magnitude and the normalized timestep to a task-conditioned LSTM, beating internal-feature baselines on RoboCasa. FAIL-Detect [30] and FIPER [24] score inputs, outputs or embeddings against successful rollouts only. FIPER alarms when two conformal scores *both* exceed their thresholds, which suppresses false alarms; we alarm when *either* crosses, as the two signals catch failures at different times. CheckVLA [17], the closest work on our benchmark, verifies each action chunk with a world model on RoboCasa365 at a conformal 5% rate of unnecessary interventions and repairs it with the same VLA. Neither FIPER nor CheckVLA compares against elapsed time.

Time, calibration and intervention. FIPER argues that late detection is not enough [24] and Hide-and-Seek weights accuracy by time [22]. Our lead is one minus FIPER's detection time and, when failures run to the limit, determines Hide-and-Seek's TWA at equal FPR (Sec. VI-A): the metric is not new, the duration-only comparator is. Lead at a fixed false-alarm budget is the accuracy–earliness trade-off of early time-series classification [29]. Split-conformal thresholds give finite-sample false-positive control under exchangeability [3]; our two-signal OR rule (Sec. IV-D) is an instance of nested conformal prediction [9], and Learn-then-Test [4] could choose its path from data with a guarantee. RESCUE [2] finds that good detection scores do not guarantee lower intervention cost, and CheckVLA couples detection to repair; an earlier alarm matters only if acting on it raises success.

III. Problem Setup

Time. The simulator runs *control steps* at RoboCasa's default 20 Hz. The policy executes each 16-step action chunk in full, so *policy inferences* $t = 1, 2, \ldots$ (0.8 s each) are our clock, and a monitor decides only at an inference. Task k's limit of T_k control steps is $H_k = \lceil T_k/16 \rceil$ inferences (29 to 272, or 22.5 to 217.5 s). Every fraction of time (t/H_k, c, ℓ) is a fraction of H_k.

Monitoring. A monitor watches a rollout of a known task k and decides at each inference t whether to alarm, using only the policy features so far, t, the task and H_k. The first alarm is at T (in inferences). FPR is the fraction of successful rollouts with any alarm. A rollout of length L ends at success or at H_k. For a failure ($L = H_k$) the *warning lead* is $\ell = (L - T)/H_k$, one minus the normalized detection time of [24], with $\ell = 0$ for a missed failure; an alarm at the final inference counts with $\ell = 0$. The mean lead $\bar{\ell}$ over failures combines recall and earliness. Detectors are compared at thresholds set on a calibration set for FPR $\leq \alpha = 0.05$.

IV. Method

A. Monitor scores

Policy score. At each inference we take the final-layer action-token hidden state of the policy's action head (1,024 dimensions). The SAFE MLP [8] maps it to a failure score $u_t^{(j)}$ for each of three training seeds j. With per-task location $m_k^{(j)}$ and scale $\sigma_k^{(j)}$ estimated on training rollouts, the policy score is

$$s_t = \frac{1}{3} \sum_{j=1}^{3} \frac{\max_{t' \leq t} u_{t'}^{(j)} - m_k^{(j)}}{\sigma_k^{(j)}}. \tag{1}$$

The running maximum makes s_t monotone, so it does not change once a threshold is crossed. On unseen tasks, $m^{(j)}$ and $\sigma^{(j)}$ are pooled over training tasks. Success-only alternatives (Mahalanobis [14], kNN [27], PCA-KMeans [16] and RND [6, 10]) are fitted on training successes and aggregated the same way.

Sequence score and input ablations. Hide-and-Seek [22] trains an LSTM with rollout labels only (in our implementation, on a 128-dimensional PCA of the same hidden state): a margin loss pushes each failure's peak score above every success's, and an onset loss acts within failures. A recurrent model can learn how long each task takes, so we retrain it unchanged on ablated inputs: *zero input*, so it sees only elapsed time; *shuffle*, a random same-task rollout's features at the same inference, so it also sees the task's typical trajectory; and *first step*, the rollout's own first-inference features, repeated, so it also sees the initial scene. Our ActProbe [11] uses chunk magnitude and the task code: its consistency signal needs overlapping chunks, and we omit its normalized-timestep input. Unlike the original methods, which threshold with time-varying conformal bands [8, 22], we threshold every score at a constant level on its running maximum (Sec. IV-D).

B. Time baselines

The *global timer* G alarms once $t \geq c\,H_k$, with one fraction $c \in (0, 1]$ for all tasks. The *per-task timer* P uses $c_k = \min(1, \rho_k + \delta)$, where ρ_k is the 95th percentile of task k's training success durations (as fractions of H_k) and the shared margin δ is set by Proposition 1. P needs successful rollouts of each task, but no failures and no policy features.

C. OR-fusion

The fused monitor (Fig. 2) alarms at

$$T = \min\{\, t : s_t > \tau_s \text{ or } t \geq c_k H_k \,\}, \tag{2}$$

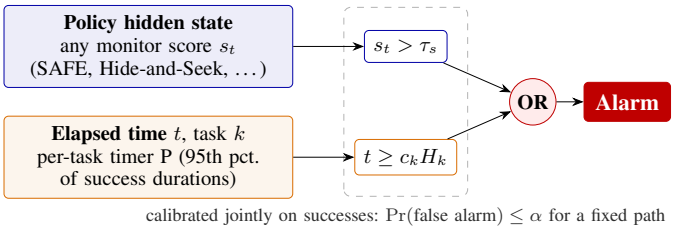


Fig. 2. Time-aware fusion: the first of an early monitor alarm and a late timer alarm wins. Split-conformal calibration on successes sets both thresholds along one path. For a path fixed independently of the calibration successes, a new success alarms with probability at most α, and once calibration certifies a usable point, every timeout is caught (Proposition 1).

with $c_k = c$ for G. Neither signal is restricted in time: the score may alarm early, and, once calibration certifies a usable point (Sec. IV-D), the timer alarms on every failure by $c_k H_k$. Any lead gain over the timer comes from failures on which the score crosses first.

D. Calibration with a finite-sample FPR guarantee

A successful rollout i of task k_i and length L_i (in inferences) raises a false alarm under $(\tau_s, c_1, \ldots, c_K)$ if and only if $M_i = \max_{t \leq L_i} s_t > \tau_s$ or $\rho_i = L_i / H_{k_i} \geq c_{k_i}$. Fix in advance a finite grid Λ and a path $\gamma(\lambda) = (\tau_s(\lambda), c_1(\lambda), \ldots, c_K(\lambda))$, $\lambda \in \Lambda$, with every coordinate nondecreasing and $c_k \leq 1$, so that a larger λ is more conservative and the alarm sets are nested. For each calibration success define

$$V_i = \min\{\lambda \in \Lambda : M_i \leq \tau_s(\lambda) \text{ and } \rho_i < c_{k_i}(\lambda)\}, \quad (3)$$

the least conservative point on the path at which rollout i stays silent, and $V_i = \infty$ if none is (e.g. a success lasting the whole limit). Because the alarm sets are nested, rollout i is silent at $\lambda \in \Lambda$ exactly when $\lambda \geq V_i$.

Proposition 1: Let n calibration successes and a new successful rollout be exchangeable, and let the path γ not depend on the calibration successes. Set $\hat{\lambda} = V_{(r)}$, the r-th smallest V_i with $r = \lceil (n+1)(1-\alpha) \rceil$, or $\hat{\lambda} = \infty$ if $r > n$. If $\hat{\lambda} = \infty$, the calibration rule cannot certify a usable point at the target and the monitor is disabled: it raises no alarm. Otherwise it runs at $\gamma(\hat{\lambda})$. Then, over calibration draws, $\Pr(\text{monitor enabled and false alarm on the new success}) \leq \alpha$.

Proof: An enabled monitor alarms on the new success if and only if $V_{n+1} > \hat{\lambda}$, and a disabled one ($\hat{\lambda} = \infty$) never does, so the event is $\{V_{n+1} > V_{(r)}\}$ with $V_{(r)} = \infty$ for $r > n$. By exchangeability, the rank of V_{n+1} among $V_1, \ldots, V_{n+1}$ is uniform (ties broken at random), so $\Pr(V_{n+1} > V_{(r)}) \leq 1 - r/(n+1) \leq \alpha$: the split-conformal quantile lemma [3]. ■

The path may use any data except calibration successes, such as development rollouts or calibration *failures*. The guarantee is per trajectory and marginal over tasks and calibration draws, not a cap on a finite test set or on each task (on the test set, one task had 4 of 10 successes alarmed); a per-task version needs 19 calibration successes per task.

Adaptive and fixed paths. Our Stage 1 results use *adaptive-path calibration*: every detector fused with P uses the path $\delta = \lambda$, $\tau_s = u + s\lambda$, with (u, s) the grid point of largest mean lead on calibration failures, each point placed at its own $\hat{\lambda}$. This choice also uses the calibration successes, so Proposition 1 bounds each candidate path, not the chosen one; over 200 re-splits of the successes, the choice raised the expected test FPR from 0.047 (P) to about 0.055. A *fixed path*, chosen independently of the calibration successes, restores the bound: chosen on the validation set, Hide-and-Seek + P still gains 0.072 at 0.061 FPR and 0.057 at matched false alarms, although the gain remains uncertain; SAFE + P gains 0.007, or 0.039 matched, as its path uses fewer false alarms than P (16 against 21). Stage 2 uses fixed paths only (Sec. VII).

V. Hypotheses

Early warning (H1). At a 5% calibration target, OR-fusing a policy-internal score with the per-task timer P increases the mean warning lead over P alone, also when P is loosened to the fused detector's realized false alarms (matched FPR).

Policy evidence (H2). At matched FPR, policy scores fused with P lead more than scores without rollout-specific information: random noise, or features or scores resampled from other rollouts of the task.

Second policy (H3). On a second VLA, RLDX-1 [12], SAFE-LSTM + P and Hide-and-Seek + P also lead P, including at matched false alarms (SAFE's MLP head adds none there).

Value of earlier warning (H4). Within a budget of $1.25\,H_k$, resetting and retrying when Hide-and-Seek + P alarms yields a higher success rate than resetting when P alarms.

Sec. VI tests H1 and H2; Stage 2 addresses H3 and H4 (Sec. VII).

VI. Experiments

A. Setup

Policy and tasks. We use the released Xiaomi-Robotics-1 (XR-1) RoboCasa365 checkpoint [28] on the 38 official RoboCasa365 tasks [19, 20] whose published success lies between 5 and 45 of 50 (10 atomic and 28 composite tasks).

Data. Disjoint sets of rollouts serve one purpose each. A *training set* (1,824 rollouts) fits every learned detector and P's ρ_k. A *validation set* (380 successes and 380 failures, 10 per class per task), with its own, smaller calibration set (3 successes and 3 failures per task), was used to develop the timers and the fused detectors. A *test set* of the same size, with new seeds, and a new calibration set of 10 successes and 10 failures per task served the tests of H1 and H2.

Unseen tasks. Each task is held out once (4 folds): monitors are retrained and thresholds calibrated without it, with OOD references and score normalization pooled over the other tasks. P needs the task's own successes, so the comparator is G. (Hide-and-Seek's evaluation [22] uses random 7/3 task splits instead.)

Baselines. We compare three families (Table I). *OOD detectors*, fitted on successful rollouts only: Mahalanobis distance [14], cosine k-NN [27], PCA-KMeans [16] and RND [10]. *Failure classifiers*, trained with rollout labels: SAFE-MLP [8], ActProbe [11] and Hide-and-Seek [22].

TABLE I

STANDALONE FAILURE DETECTION WITH XR-1 (TEST SET, 380 + 380 ROLLOUTS) AT THRESHOLDS CALIBRATED FOR 5% FPR. SINGLE MODELS: MEAN ± STD ACROSS 3 TRAINING SEEDS; **BOLD**/UNDERLINE: BEST/SECOND BEST.

	Policy: XR-1 (Success Rate: 55.6%)				
		Seen tasks		Unseen tasks	
	Method	bACC↑	TWA↑	bACC↑	TWA↑
OOD	Mahalanobis	$0.818^{\pm 0.001}$	$0.582^{\pm 0.001}$	$0.726^{\pm 0.000}$	$0.543^{\pm 0.000}$
OOD	Cosine k-NN	$0.788^{\pm 0.000}$	$0.577^{\pm 0.000}$	$0.729^{\pm 0.000}$	$0.544^{\pm 0.000}$
OOD	PCA-KMeans	$0.796^{\pm 0.001}$	$0.578^{\pm 0.001}$	$0.728^{\pm 0.001}$	$0.542^{\pm 0.001}$
OOD	RND	$0.793^{\pm 0.002}$	$0.572^{\pm 0.001}$	$0.728^{\pm 0.001}$	$0.547^{\pm 0.001}$
Class.	SAFE-MLP	$0.763^{\pm 0.003}$	$0.582^{\pm 0.011}$	$0.713^{\pm 0.003}$	$0.541^{\pm 0.001}$
Class.	ActProbe	$0.654^{\pm 0.003}$	$0.558^{\pm 0.006}$	$0.500^{\pm 0.000}$	$0.500^{\pm 0.001}$
Class.	Hide-and-Seek	$0.742^{\pm 0.033}$	$\mathbf{0.599}^{\pm 0.005}$	$\underline{0.790}^{\pm 0.002}$	$\mathbf{0.595}^{\pm 0.005}$
Time	Global timer G*	**0.988**	0.550	**0.988**	<u>0.551</u>
Time	Per-task timer P*	<u>0.972</u>	<u>0.585</u>	–	–

*Deterministic. P needs the task's own successful rollouts.

Elapsed time, with no policy access: G and P. Token-uncertainty baselines need discrete action tokens and STAC [1] overlapping chunks, which XR-1 does not produce.

Metrics. Following [24, 22], at thresholds calibrated for FPR ≤ 0.05 we report test FPR, balanced accuracy (bACC; equal to weighted accuracy on our balanced sets), time-weighted accuracy $\mathrm{TWA} = \frac{1}{2}(\bar{\ell} + 1 - \mathrm{FPR})$ and detection time $\mathrm{DT} = 1 - \bar{\ell}$, a miss counting as 1 (FIPER averages DT over detected failures only). As in [22], each learned detector is trained with 3 random seeds, and tables give the mean ± std of single models across these training seeds. For H1 and H2 we use *three-seed ensembles*, which average the 3 models' scores. *Matched* gains compare with P loosened on the test set to the fused detector's false alarms, which favors the timer. For the two central matched gains, 95% bootstrap CIs resample the test set's tasks, then failures within each task (2,000 resamples, paired across methods). They are conditional on the fitted thresholds and the matched timer, so they exclude calibration and matching uncertainty.

B. Standalone failure detection

Evaluated on their own, as in prior work (Table I), the OOD detectors and failure classifiers catch 36% to 69% of the failures on seen tasks at 5% FPR. The two timers, which never read the policy, catch all of them (bACC 0.97 and 0.99, against at most 0.82). The global timer alarms last, but P (DT 0.775) alarms earlier than every learned detector except Hide-and-Seek (0.754), which alone exceeds its TWA. On unseen tasks, Hide-and-Seek is the only detector above the global timer's TWA.

C. Warning lead over the per-task timer

SAFE: a small gain (H1, H2). On the test set, the SAFE ensemble fused with P leads P by 0.030 of the time limit at 0.061 FPR. At the same target, random and within-task resampled scores gain at most 0.009, so the gain is not an artifact of the fusion itself. At matched false alarms, however, it shrinks to 0.015 (95% CI $[-0.007, 0.041]$).

TABLE II

WARNING LEAD ON THE TEST SETS. MONITORS ARE OR-FUSED AT A 5% FPR TARGET WITH P ON SEEN TASKS AND G ON UNSEEN TASKS, USING ADAPTIVE-PATH CALIBRATION (SEC. IV-D; NOT COVERED BY PROPOSITION 1). $\Delta\bar{\ell}$: GAIN IN MEAN LEAD OVER THAT TIMER. SINGLE MODELS: MEAN ± STD ACROSS 3 TRAINING SEEDS.

	Policy: XR-1 (Success Rate: 55.6%; 380 + 380 test rollouts)					
		Seen tasks + P			Unseen tasks + G	
	Method	FPR↓	$\Delta\bar{\ell}\uparrow$	TWA↑	FPR↓	$\Delta\bar{\ell}\uparrow$	TWA↑
Time	Timer G	0.024	-0.102	0.550	0.024	–	0.551
Time	Timer P	0.055	–	0.585	–	–	–
OOD	Mahalanobis	0.055	$\underline{0.034}^{\pm 0.000}$	$\underline{0.602}^{\pm 0.000}$	0.073	$0.044^{\pm 0.001}$	$0.549^{\pm 0.001}$
OOD	Cosine k-NN	0.058	$0.021^{\pm 0.000}$	$0.594^{\pm 0.000}$	0.079	$\underline{0.053}^{\pm 0.000}$	$0.550^{\pm 0.000}$
OOD	PCA-KMeans	0.060	$0.022^{\pm 0.001}$	$0.594^{\pm 0.000}$	0.071	$\underline{0.053}^{\pm 0.001}$	$\underline{0.554}^{\pm 0.000}$
OOD	RND	0.060	$0.021^{\pm 0.001}$	$0.593^{\pm 0.000}$	0.073	$0.050^{\pm 0.001}$	$0.551^{\pm 0.000}$
Cl.	SAFE-MLP	0.058	$0.024^{\pm 0.015}$	$0.596^{\pm 0.006}$	0.053	$0.028^{\pm 0.009}$	$0.551^{\pm 0.005}$
Cl.	Hide&Seek	0.055	$\mathbf{0.039}^{\pm 0.006}$	$\mathbf{0.605}^{\pm 0.005}$	0.059	$\mathbf{0.122}^{\pm 0.005}$	$\mathbf{0.595}^{\pm 0.004}$
Abl.	First step	0.055	$0.001^{\pm 0.001}$	$0.586^{\pm 0.001}$	0.027	$0.002^{\pm 0.003}$	$0.550^{\pm 0.001}$
Abl.	Zero input	0.062	$0.013^{\pm 0.003}$	$0.588^{\pm 0.000}$	0.037	$0.020^{\pm 0.008}$	$0.554^{\pm 0.005}$
Abl.	Shuffle	0.058	$0.005^{\pm 0.001}$	$0.586^{\pm 0.002}$	–	–	–
	Policy: RLDX-1 (Success Rate: 56.2%; 100 + 100 recorded test rollouts)						
		Seen tasks + P			Unseen tasks + G		
	Method	FPR↓	$\Delta\bar{\ell}\uparrow$	TWA↑	FPR↓	$\Delta\bar{\ell}\uparrow$	TWA↑
Time	Timer G	0.010	-0.239	0.511	0.000	–	0.513
Time	Timer P	0.030	–	0.621	–	–	–
OOD	Mahalanobis	0.040	$0.030^{\pm 0.000}$	$0.631^{\pm 0.000}$	0.030	$0.112^{\pm 0.000}$	$0.554^{\pm 0.000}$
OOD	Cosine k-NN	0.050	$0.005^{\pm 0.000}$	$0.613^{\pm 0.000}$	0.010	$0.112^{\pm 0.000}$	$\underline{0.564}^{\pm 0.000}$
OOD	PCA-KMeans	0.047	$0.029^{\pm 0.004}$	$0.627^{\pm 0.001}$	0.040	$\underline{0.115}^{\pm 0.004}$	$0.550^{\pm 0.002}$
OOD	RND	0.040	$0.015^{\pm 0.007}$	$0.623^{\pm 0.010}$	0.037	$0.099^{\pm 0.013}$	$0.544^{\pm 0.011}$
Class.	SAFE-MLP	0.053	$0.008^{\pm 0.012}$	$0.613^{\pm 0.007}$	0.070	$0.044^{\pm 0.000}$	$0.500^{\pm 0.000}$
Class.	SAFE-LSTM	0.033	$\underline{0.036}^{\pm 0.003}$	$\underline{0.637}^{\pm 0.003}$	0.067	$0.101^{\pm 0.006}$	$0.530^{\pm 0.000}$
Class.	Hide&Seek	0.050	$\mathbf{0.074}^{\pm 0.030}$	$\mathbf{0.648}^{\pm 0.004}$	0.057	$\mathbf{0.196}^{\pm 0.013}$	$\mathbf{0.583}^{\pm 0.021}$
Abl.	First step	0.050	$-0.004^{\pm 0.014}$	$0.609^{\pm 0.005}$	0.090	$0.038^{\pm 0.025}$	$0.487^{\pm 0.012}$
Abl.	Zero input	0.037	$0.012^{\pm 0.010}$	$0.623^{\pm 0.002}$	0.023	$0.010^{\pm 0.024}$	$0.506^{\pm 0.008}$
Abl.	Shuffle	0.030	$0.007^{\pm 0.005}$	$0.624^{\pm 0.002}$	–	–	–

Mean lead of the timers: XR-1 P 0.225, G 0.123 (seen) and 0.126 (unseen); RLDX-1 P 0.271, G 0.032 and 0.026. Unseen FPRs can exceed 0.05, as thresholds are set on the seen tasks.

Hide-and-Seek: a larger gain (H1, H2). The Hide-and-Seek ensemble fused with P leads P by 0.071 at the same 0.061 FPR (random and resampled scores: at most 0.012). At matched false alarms it gains 0.056 (95% CI $[-0.014, 0.120]$), and its point estimates are positive at every target from 2% to 10%, where SAFE's average 0.013 (Fig. 3b). Its advantage over SAFE + P (0.041) is uncertain. A single Hide-and-Seek model gains only 0.039 ± 0.006 (Table II), so the ensemble is part of its warning.

Where the gain comes from. All six policy scores gain more on the 10 atomic tasks (0.045 to 0.160) than on the 28 composite ones (0.008 to 0.039; Fig. 3a). Hide-and-Seek + P alarms later than P on 16 tasks, where its looser timer margin spends the false-alarm budget on the score, and five tasks carry 78% of its gain.

D. Is the extra warning learned elapsed time?

A recurrent detector trained on rollouts that all fail at the limit could learn task durations, and the zero-input ablation does separate failures as well as Hide-and-Seek (Sec. I). On the test set, however, the first-step, zero-input and shuffle ablations gain -0.003, -0.003 and -0.011 over P at matched false alarms, against 0.056 for Hide-and-Seek (and 0.070, 0.059 and

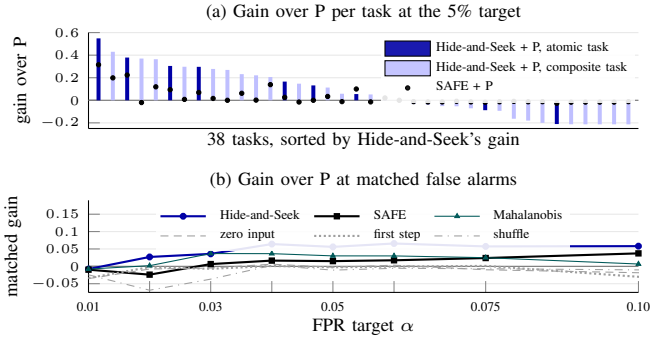


Fig. 3. Where the gain over the per-task timer P comes from (XR-1, test set). (a) Atomic tasks gain about four times as much as composite ones (Hide-and-Seek: 0.160 vs. 0.039 on average). (b) With P loosened to the fused detector's false alarms, Hide-and-Seek's point estimates are positive at every target from 2% to 10% (at 5%: 95% CI $[-0.014, 0.120]$); its input ablations never gain more than 0.01.

0.060 less than it at the same target): its warning develops during the rollout and is neither elapsed time nor the task's typical trajectory.

E. Unseen tasks, second policy and second benchmark

Unseen tasks (Tables I and II). With no rollout of the task, as in SAFE's setting [8], only Hide-and-Seek keeps a gain over G on XR-1: 0.127 as an ensemble (zero-input ablation: 0.033), 0.111 more than SAFE. SAFE's ensemble gain of 0.017 is no larger than the best of 20 random scores (0.019; single models: 0.028), and against G loosened to its false alarms it is -0.079 (Hide-and-Seek: $+0.051$). The OOD detectors' gains cost three times G's false alarms. Once 1 to 5 successes of the new task exist, a per-task timer can be built, and Hide-and-Seek still adds 0.088 to 0.099 over it (SAFE about 0.01).

Second policy. On recorded RLDX-1 rollouts [12] (20 tasks; Table II, bottom), all failures again ran to the time limit. As ensembles, SAFE-LSTM + P gains 0.034 and Hide-and-Seek + P, with the XR-1 settings, 0.083 (0.052 at matched false alarms). Both gains come from atomic tasks: on composite tasks no score adds clear lead (Hide-and-Seek 0.010). On unseen tasks, Hide-and-Seek again gains most over G (0.196 ± 0.013; ablations ≤ 0.038); SAFE-LSTM and the OOD detectors (default settings) gain 0.10 to 0.115 over this late timer (lead 0.026).

FIPER's benchmark. In FIPER's released rollouts [24] (five tasks, two on a real robot), length alone separates failures with ROC-AUC 0.92–1.00 in four tasks. Re-running FIPER's code (reproduced within 0.05) with elapsed time as one more method, the timer has the best balanced accuracy under FIPER's selection rule (0.81 against 0.78) at 0.09 FPR, against 0.35 for FIPER. At FIPER's own FPR (0.06 to 0.56 by task), however, FIPER warns 0.146 of the episode earlier than the timer (from 0.019 on PushChair to 0.385 on Pretzel, both real): the test credits early warning wherever it occurs.

F. When do failures become irreversible?

One annotator, blind to alarms, marked the point of no return (the first state from which the task could not be completed without outside help) on 48 test-set failures: 38 (79%) reached no such state before the limit (*recoverable*), most often because the robot was slow or nearly succeeded; a second annotator agreed on 11 of 12 ($\kappa = 0.80$). On all 10 failures that did become irreversible, P alarmed after the point of no return, and no detector flags more than 8% of failures by half the limit.

VII. Stage 2: Planned Experiments

Second policy (E1). We test H3 on new RLDX-1 rollouts: fresh calibration and test sets of 20 successes and 20 failures for each of its 20 tasks (about 2,500 rollouts). The detectors (SAFE-LSTM, Hide-and-Seek), H_k, ρ_k and each fused path are fixed on the existing rollouts, so the new calibration successes set only $\hat{\lambda}$ and Proposition 1 applies to the evaluated detector. We compare each with P, at the same target and at matched false alarms, and with random-score fusions.

Reset and retry (E2). We evaluate H4 in closed loop on 1,000 RLDX-1 scenes (50 per task) with a shared budget of $1.25\,H_k$. Each scene runs once without reset, which also gives the no-reset outcome (in a pilot, it rescued only 4% of RLDX-1 failures); for each monitor, we branch from this run at its first alarm, restore the scene and retry in the remaining budget. Monitor inference latency is charged against this budget, and both monitors use identical reset and retry accounting. We report the paired success-rate difference between Hide-and-Seek + P and P resets on the same scenes, with a 95% CI, together with reset against no reset, unnecessary resets and wasted time.

Further analyses (E3). Action-only and multi-sample monitors, richer time baselines, unseen tasks on new rollouts, and lead from the point of no return (250 to 500 failures).

VIII. Conclusion and Limitations

When failure means running out of time, a monitor must beat a timer at the same false-alarm rate. Against a per-task timer at the same target FPR, three-seed ensembles of SAFE and Hide-and-Seek added 0.030 and 0.071 of the time limit on new test rollouts; at matched false alarms these shrink to 0.015 and 0.056, and both remain uncertain. Input ablations trace Hide-and-Seek's warning to the rollout, and on XR-1 it carries over to unseen tasks, where SAFE's does not. Stage 2 tests both monitors on a second policy and measures their value in closed-loop retry.

Limitations. Alarms were recorded, not acted on, in simulation, and lead is measured to the time limit, not to an irreversible failure. RLDX-1 is evaluated only on recorded rollouts. The timers and fusions were designed on the validation set, and the matched, per-task, unseen-task and FIPER analyses were added after the test set was scored; Hide-and-Seek, the better of two supervised baselines on the validation set, was tested on a test set already used for SAFE. Monitors assume a known task and time limit; fixed-horizon data need failure-time labels.

References

[1] Christopher Agia et al. Unpacking failure modes of generative policies: Runtime monitoring of consistency and progress. In *Conference on Robot Learning (CoRL)*, volume 270 of *PMLR*, 2024.

[2] David Ahmedt-Aristizabal et al. RESCUE: Is this alarm worth acting on? Counterfactual evaluation of VLA runtime monitors, 2026. arXiv:2610.13516.

[3] Anastasios N. Angelopoulos and Stephen Bates. A gentle introduction to conformal prediction and distribution-free uncertainty quantification, 2021. arXiv:2107.07511.

[4] Anastasios N. Angelopoulos et al. Learn then test: Calibrating predictive algorithms to achieve risk control. *Annals of Applied Statistics*, 2025. arXiv:2110.01052.

[5] Kevin Black et al. π_0: A vision-language-action flow model for general robot control. In *Robotics: Science and Systems (RSS)*, 2025.

[6] Yuri Burda et al. Exploration by random network distillation. In *International Conference on Learning Representations (ICLR)*, 2019.

[7] Robert Geirhos et al. Shortcut learning in deep neural networks. *Nature Machine Intelligence*, 2(11):665–673, 2020.

[8] Qiao Gu et al. SAFE: Multitask failure detection for vision-language-action models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

[9] Chirag Gupta, Arun K. Kuchibhotla, and Aaditya Ramdas. Nested conformal prediction and quantile out-of-bag ensemble methods. *Pattern Recognition*, 127:108496, 2022.

[10] Nantian He et al. ReDiffuser: Reliable decision-making using a diffuser with confidence estimation. In *International Conference on Machine Learning (ICML)*, volume 235 of *PMLR*, pages 17921–17933, 2024.

[11] Bingjia Huang et al. ActProbe: Action-space probe for early failure detection of generative robot policies, 2026. arXiv:2606.08508.

[12] Dongyoung Kim et al. RLDX-1 technical report, 2026. arXiv:2605.03269.

[13] Moo Jin Kim et al. OpenVLA: An open-source vision-language-action model. In *Conference on Robot Learning (CoRL)*, 2024.

[14] Kimin Lee et al. A simple unified framework for detecting out-of-distribution samples and adversarial attacks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.

[15] Bo Liu et al. LIBERO: Benchmarking knowledge transfer for lifelong robot learning. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2023.

[16] Huihan Liu et al. Multi-task interactive robot fleet learning with visual world models. In *Conference on Robot Learning (CoRL)*, volume 270 of *PMLR*, 2024.

[17] Yushan Liu et al. CheckVLA: Execution-time verification with action-conditioned world model for long-horizon mobile manipulation, 2026. arXiv:2607.26789.

[18] Jie Ma, Zongxi Liu, and Yi Zhu. FailureSpot: Label-efficient timestamp-level failure detection for vision-language-action models, 2026. arXiv:2609.04277.

[19] Soroush Nasiriany et al. RoboCasa: Large-scale simulation of everyday tasks for generalist robots. In *Robotics: Science and Systems (RSS)*, 2024.

[20] Soroush Nasiriany et al. RoboCasa365: A large-scale simulation framework for training and benchmarking generalist robots. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2603.04356.

[21] Fabio Pardo, Arash Tavakoli, Vitaly Levdik, and Petar Kormushev. Time limits in reinforcement learning. In *International Conference on Machine Learning (ICML)*, volume 80 of *PMLR*, pages 4045–4054, 2018.

[22] Seongheon Park et al. Hide-and-seek in trajectories: Discovering failure signals for VLA runtime monitoring. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2026. arXiv:2605.30834.

[23] Harshitha Rajaprakash et al. SAFECAST: Robust failure detection for VLA policies with contrast-set training and calibration, 2026. arXiv:2608.04246.

[24] Ralf Römer et al. Failure prediction at runtime for generative robot policies. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

[25] Florian Seligmann et al. VLA-FAIL: Efficient task failure detection for finetuned vision-language-action models. In *Conference on Robot Learning (CoRL)*, 2026.

[26] Michael Soddy et al. Predictable before action, resistant to correction: Failure signals in vision-language-action models. In *NeurIPS Workshop on From Pretrained Representations to Acting Agents (PTA)*, 2026.

[27] Yiyou Sun et al. Out-of-distribution detection with deep nearest neighbors. In *International Conference on Machine Learning (ICML)*, 2022.

[28] Xiaomi Robotics Team. Xiaomi-Robotics-1: Scaling vision-language-action models with over 100K hours of real-world trajectories, 2026. arXiv:2607.15330.

[29] Zhengzheng Xing, Jian Pei, and Philip S. Yu. Early classification on time series. *Knowledge and Information Systems*, 31(1):105–127, 2012. doi: 10.1007/s10115-011-0400-x.

[30] Chen Xu et al. Can we detect failures without failure data? Uncertainty-aware runtime failure detection for imitation learning policies. In *Robotics: Science and Systems (RSS)*, 2025.